



SHORT ANSWER TEST

COURSE NAME: Software Testing

COURSE CODE: CSA3711

Code of Conduct:

I (PRADEEP.S /192521024) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Pradeep.s

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Testing as an Engineering Activity

Software testing should be treated as a systematic engineering activity rather than random execution of test cases. The team should first study requirements, identify risks, define test objectives, design test cases using techniques such as equivalence partitioning and boundary value analysis, execute them, and record defects.

Structured test design gives repeatable and measurable testing. It improves coverage because normal, boundary, invalid, and exceptional conditions are deliberately considered. For example, if an age field accepts 18–60, tests should include 17, 18, 60 and 61. This systematic approach increases defect detection and software quality.

2. Well-Defined Testing Process

A well-defined testing process provides planned and controlled testing activities from requirements through maintenance. In a banking system, testers can define requirements, prepare test plans, design functional and security test cases, execute them, report defects, perform regression testing, and give a clear test summary.

For example, before releasing a money-transfer feature, testers verify valid transfers, insufficient balance, invalid account numbers, duplicate transactions and transaction failures. A standard process improves consistency, traceability, coverage and reliability, reducing the possibility of serious production failures.

3. Testing as a Structured SDLC Process

Testing should be performed throughout the SDLC rather than only after coding. During requirements, testers review requirements for ambiguity. During design, they review interfaces and prepare test conditions. During coding, unit testing is performed. During integration, interactions between modules are tested. During system testing, the complete application is tested against requirements. Finally, regression and acceptance testing are performed before release.

Process discipline ensures that defects are detected at the appropriate stage, test activities are documented, and every requirement is verified. This reduces integration and system-level defects and improves overall quality.

4. Error, Defect and Failure

An error is a human mistake made during requirements, design or coding. A defect is the flaw introduced into the software because of that error. A failure occurs when the executing software

For example, a developer may misunderstand that a password must contain eight characters; this misunderstanding is an error. The incorrect validation condition written in the login program is a defect. When a valid eight-character password is rejected during execution, the observed incorrect behavior is a failure. Thus, an error can introduce a defect, and a defect can cause a failure when executed.

5. Early Defect Detection

The principle of early testing should be applied. Testing activities should begin as early as possible in the SDLC, starting with requirement reviews and continuing through design, coding and execution.

For example, if a healthcare requirement says 'patient data should be protected' without defining the required security controls, a tester can identify the ambiguity during requirement review. Correcting it at that stage is much cheaper than correcting implemented software later. Early detection reduces rework, cost and schedule delays and improves testing effectiveness.

6. Tester Involvement in Requirements

A tester should participate in requirement analysis by reviewing requirements for correctness, completeness, consistency, feasibility, testability and ambiguity. The tester can identify missing conditions and clarify expected results before development starts.

In a healthcare system, a requirement such as 'patient information should be available quickly' is ambiguous unless response time is specified. The tester can ask for a measurable requirement such as a maximum response time. Early involvement prevents misunderstandings from becoming design and implementation defects and makes requirements easier to test.

7. Requirement Defects in the Lifecycle

Defects can originate when requirements are incomplete, ambiguous, inconsistent or incorrectly understood. If an unclear payroll requirement says 'calculate overtime correctly' without defining overtime rules, developers may implement different assumptions. The resulting program may calculate incorrect salaries.

Clear requirements should define inputs, business rules, calculations, exceptions and expected outputs. Requirements should also be reviewed with stakeholders and testers for ambiguity and consistency. Clear and testable requirements prevent defects from propagating into design, coding and later testing stages.

8. Boundary Value Defect

A crash at boundary input values is typically a boundary-related functional defect caused by inadequate boundary testing. Testing only normal values may miss errors that occur at the minimum, maximum, just below, and just above permitted limits.

For example, if an account withdrawal amount is allowed from ₹100 to ₹10,000, tests should include ₹99, ₹100, ₹101, ₹9,999, ₹10,000 and ₹10,001. Boundary Value Analysis would expose incorrect comparison or validation logic before release and could therefore prevent the crash.

9. Defect Repository

A defect repository stores information about previously detected defects, including defect description, module, severity, cause, status and resolution. Testers can analyze this history when designing new test cases.

If a payment module repeatedly had rounding defects, future regression tests should specifically include different amounts, decimal values and boundary amounts. The repository therefore helps identify defect-prone areas and recurring patterns. Reusing this information improves test case design, regression coverage and software quality.

10. ATM Functional Defect

An ATM withdrawal problem caused by incorrect transaction logic is a functional defect because the software does not perform the required business function correctly. For example, the ATM may deduct money from the account without dispensing cash, or may allow a withdrawal exceeding the available balance.

It could be prevented through requirement-based functional test cases covering valid withdrawal, insufficient balance, invalid amount, daily limits, cancellation, network failure and successful cash dispensing. Integration and end-to-end testing should also verify that account and ATM transaction states remain consistent.

11. Defect Clustering

Defect clustering means that a small number of modules often contain a large proportion of the defects. Testers should analyze historical defect data and identify modules with high defect density.

For example, if the payment and authentication modules contain most defects, testing effort should prioritize these modules with deeper functional, boundary, security and regression testing. This does not mean other modules are ignored; rather, risk and historical evidence are used to allocate effort efficiently. It improves defect detection with limited testing resources.

12. Performance Defect

Slow performance under heavy user load is a non-functional performance defect. The software may provide correct results but fail to meet required response time, throughput or scalability expectations.

The team should reproduce the problem using load and stress testing, measure response time and resource usage, identify the bottleneck, and fix and retest it. Proper classification directs the team toward suitable techniques and tools instead of treating the issue only as a functional defect.

13. Historical Data for Regression Testing

Historical defect data can be used to identify modules, functions and scenarios that previously failed. These cases should be added or strengthened in the regression test suite and executed after every relevant change.

For example, if previous releases had repeated login and payment defects, those tests should be prioritized during regression. Historical data helps focus testing on risky areas, increases regression coverage and avoids repeating old mistakes while reducing unnecessary test execution.

14. Tester Role in Integration Quality

A tester helps minimize integration defects by understanding module interfaces, data flow and dependencies early. The tester should participate in discussions between developers and testers, review interface requirements, identify integration risks and prepare integration test cases.

For example, if one module sends date data in one format and another expects a different format, early

review can identify the mismatch before integration. Early tester involvement improves communication, detects interface defects sooner and reduces expensive integration failures.

15. Defect Prevention

Defect prevention focuses on finding and removing the causes of recurring defects rather than repeatedly fixing the same symptoms. The team should analyze defect trends, perform root-cause analysis, improve requirements and coding standards, conduct reviews, and update test cases.

If every release contains the same input-validation defect, the team should determine why validation was missed and introduce a standard validation checklist and reusable tests. Prevention reduces recurrence, maintenance effort and defect leakage, thereby improving long-term software quality.

16. Standardized Testing Strategy

The mobile application should use a standardized testing strategy with defined test levels, common test case formats, entry and exit criteria, defect reporting rules and regression procedures. Requirements should be mapped to test cases and important device, OS and network conditions should be covered.

A structured approach avoids inconsistent testing between team members and releases. Reusable regression suites, risk-based priorities and documented results improve reliability and make testing repeatable and measurable.

17. Testing vs Debugging

Testing and debugging are different activities. Testing is performed to reveal failures or demonstrate that the software does not meet requirements. Debugging is the development activity of locating the cause of a defect and correcting the code.

For example, if an online shopping application calculates the wrong total, a tester reports the failure with input and expected output. The developer then debugs the program, traces the calculation logic, finds the incorrect formula and fixes it. Testing detects the problem; debugging finds and removes its cause.

18. Cross-Browser Testing

A cross-browser testing strategy should test the web application on the browsers and versions that are supported by the product. Testers should cover important functions, layouts, JavaScript behavior, forms, navigation and responsive behavior on each target browser.

For example, the checkout process should be executed on supported Chrome, Edge, Firefox and mobile browsers. Differences should be recorded with browser, version, operating system, steps and screenshots. Fixes should then be verified and regression tested. This reduces browser-specific failures and improves customer experience.

19. Validation Gaps

The logistics application should have clear validation rules for inputs, calculations, data formats and expected outputs. Testers should identify missing validation by comparing requirements with test conditions and by testing valid, invalid, boundary and unexpected inputs.

For example, a delivery distance field should reject negative values and invalid formats. Proper validation prevents incorrect data from entering later processing stages. Requirement-based and boundary testing therefore improve reliability and help detect defects early.

20. Systematic Test Design

Random testing alone cannot guarantee adequate coverage. Testing as an engineering activity requires requirements analysis, risk identification, systematic test design, execution and defect reporting.

The team should derive test cases from requirements and use techniques such as equivalence partitioning, boundary value analysis, decision tables and state-based testing where appropriate. Test cases should include positive, negative and edge conditions. Systematic design provides traceability and better coverage, resulting in stronger defect detection than unplanned random tests.

21. Integration Data Consistency

Data inconsistency between healthcare modules should be investigated by checking interfaces, data formats, field mappings, database transactions and synchronization rules. Testers should identify where the same data changes incorrectly while moving between modules.

For example, if a patient's date of birth is stored correctly in registration but displayed incorrectly in billing, integration tests should compare the data at both interfaces. Contract/interface tests, end-to-end tests and database verification can detect the mismatch. Proper integration testing ensures consistent data across modules.

22. Tester Role Across the SDLC

Testers should contribute throughout the SDLC. In requirements, they review testability and ambiguity. In design, they identify integration and risk areas. During coding, they support unit-test expectations and prepare test cases. During integration and system testing, they execute tests and report defects. During maintenance, they perform regression testing.

Early involvement allows defects to be prevented rather than discovered only after implementation. It improves communication, reduces rework and helps ensure that the final product satisfies requirements.

23. Peak-Usage Failure and Coverage

The failure indicates insufficient test coverage for high-load or peak-usage scenarios. The team should identify realistic user-load conditions and include performance, stress and capacity scenarios in the test plan.

For example, if the system normally supports 1,000 users but may receive 5,000 users during peak time, testing should include the expected peak and appropriate stress conditions. Combining functional coverage with performance scenarios helps reveal failures that normal functional testing may miss and improves release reliability.

24. Defect Documentation and Repository

A defect repository should contain a unique defect ID, title, detailed description, steps to reproduce, expected result, actual result, severity, priority, environment, status, owner and resolution. This makes defects easy to track throughout their lifecycle.

Good documentation helps developers reproduce and fix problems, allows testers to verify fixes, and enables managers to analyze defect trends. Historical records can also be reused for regression testing and defect prevention. Thus, proper documentation improves traceability and analysis.

25. Banking Transaction Failures

The banking system should use a structured, risk-based testing process instead of inconsistent testing. Requirements should be reviewed, transaction scenarios should be identified, test cases should cover positive, negative, boundary and failure conditions, and integration and regression testing should be performed.

Important cases include valid transfers, insufficient funds, invalid accounts, duplicate transactions, timeout, network failure and rollback. Defects should be recorded and analyzed for root causes. Structured testing improves transaction reliability, consistency and confidence before release.

26. Edge Case and Boundary Testing

Testing only normal inputs leaves important edge conditions untested. Testers should identify minimum, maximum, just-inside and just-outside values, as well as empty, null, zero and unexpected inputs where applicable.

For example, for a field accepting 1–100, tests should include 0, 1, 2, 99, 100 and 101. Boundary Value Analysis is effective because defects frequently occur near limits. Including edge cases improves defect detection and prevents failures caused by incorrect boundary conditions.

27. Process-Based Testing

A process-based approach defines common testing procedures, standards, templates, review steps, entry and exit criteria, defect reporting rules and regression practices for all modules. Testers should follow the same basic process while adapting test cases to module-specific risks.

Standardization ensures that important activities are not skipped and that results are comparable across modules. It improves consistency, traceability, repeatability and overall quality, reducing uneven testing outcomes.

28. Unexpected Input Validation

The web application should validate inputs at appropriate boundaries and test both expected and unexpected values. Test cases should include invalid formats, missing values, extreme values, special characters, unexpected data types and values outside permitted ranges.

For example, if a quantity field accepts positive integers, negative numbers, decimals, text and empty input should be tested. Proper input validation prevents invalid data from reaching business logic and improves robustness, reliability and security.

29. Defect Leakage

Frequent production defect leakage may result from incomplete requirements, inadequate test coverage, weak test design, insufficient regression testing, poor environment coverage or inadequate defect analysis. The team should analyze escaped defects to identify their root causes.

Risk-based testing, early reviews, boundary and negative testing, stronger regression suites and production-like test environments can reduce leakage. Historical escaped defects should be converted into regression tests. This creates continuous improvement and prevents similar defects from reaching production again.

30. Early Defect Detection Strategy

The team should shift testing left by involving testers early and applying reviews and static analysis before execution testing. Requirements should be reviewed for ambiguity, designs should be inspected, and developers should perform unit testing. Integration, system and regression testing should then follow systematically.

For example, finding an incorrect requirement during review is much cheaper than discovering it after deployment. Early detection reduces rework, testing cost and schedule delays while improving product quality. A structured, risk-based strategy therefore increases efficiency and defect detection effectiveness.